



Projet d'Ingénierie de Réseau

Modélisation, Prédiction et Analyse de Robustesse du Trafic Réseau Satellite

Thomas NIELLEN

Mathis PATEAU

Arthur SAUVEZIE

Tristant MOREAU

Alexandre MOHIB

Filière Télécommunications et Réseaux

ENSEEIH – Toulouse INP

24 mai 2026

Résumé

Ce rapport présente une étude comparative de différents modèles de prédiction temporelle pour le trafic réseau dans un contexte de télécommunications par satellite. L'objectif est d'optimiser l'allocation dynamique de la bande passante en anticipant la demande de trafic. Nous confrontons des approches statistiques traditionnelles (Autorégressif AR, ARIMA) à des méthodes d'apprentissage automatique (Random Forest, Support Vector Regression SVR) et d'apprentissage profond (LSTM, GRU). Afin de tester la résilience opérationnelle des algorithmes, nous réalisons une analyse de stabilité par simulation de Monte Carlo avec injection de bruit gaussien. Nos résultats montrent que si les réseaux de neurones récurrents excellent en précision nominale, les méthodes d'ensemble et de frontières de décision offrent une robustesse bien supérieure face aux perturbations de mesure.

Table des matières

1	Introduction	3
1.1	Contexte et Enjeux	3
1.2	Objectifs du Projet	3
2	État de l'Art des Algorithmes et Modèles	3
2.1	Modèles Statistiques Linéaires	3
2.1.1	Modèle Autorégressif (AR)	3
2.1.2	Modèle ARIMA	3
2.2	Modèles d'Apprentissage Machine (Machine Learning)	4
2.2.1	Forêt Aléatoire (Random Forest Regression)	4
2.2.2	Régression par Vecteurs de Support (SVR)	4
2.3	Modèles d'Apprentissage Profond (Deep Learning)	4
2.3.1	Réseaux Récurrents LSTM	4
2.3.2	Gated Recurrent Unit (GRU)	4
3	Implémentation Logicielle et Protocole Expérimental	5
3.1	Architecture globale du code	5
3.2	Prétraitement des Données	5
3.3	Protocole d'Évaluation de Monte Carlo	5
3.4	Détails d'Implémentation par Modèle	6
3.4.1	Modèles Statistiques Linéaires (AR et ARIMA)	6
3.4.2	Modèle d'Apprentissage Machine : Random Forest	6
3.4.3	Modèle d'Apprentissage Machine : Support Vector Regression (SVR)	6
3.4.4	Modèles d'Apprentissage Profond (LSTM et GRU)	7
4	Analyse des Résultats	7
4.1	Métriques d'Évaluation	7
4.2	Scénarios d'Évaluation et Modes d'Exécution	8
4.3	Performances en Mode 1 (Analyse Classique)	8
4.4	Performances en Mode 2 (Comparaison Globale)	10
4.5	Analyses Croisées : Voies (Mode 3) et Faisceaux (Mode 4)	11
4.6	Robustesse au Bruit : Analyse de Monte Carlo (Mode 5)	12
4.6.1	Comportement d'ARIMA sous perturbation	12
4.6.2	Sensibilité des Modèles de Deep Learning	12
4.6.3	Stabilité du Random Forest et de SVR	13
5	Conclusion et Perspectives	13
5.1	Synthèse des travaux	13
5.2	Perspectives futures	13

1 Introduction

1.1 Contexte et Enjeux

Le développement des constellations de satellites en orbite basse (LEO) et moyenne (MEO), ainsi que l'évolution des satellites géostationnaires à haut débit (HTS), ont transformé le paysage des télécommunications spatiales. Contrairement aux réseaux terrestres câblés, la ressource orbitale en bande passante (fréquence, puissance) est limitée, coûteuse et soumise à des variations dynamiques importantes (conditions météorologiques, déplacements de terminaux, variations d'usages).

L'allocation statique de ressources mène inévitablement à des inefficacités :

- **Sous-allocation** : congestion du réseau, augmentation de la latence, pertes de paquets et dégradation de la Qualité de Service (QoS).
- **Sur-allocation** : gaspillage d'une ressource spectrale précieuse et surcoût économique majeur pour l'opérateur.

1.2 Objectifs du Projet

L'objectif de ce projet est de concevoir un système d'allocation dynamique de bande passante basé sur la prédiction de trafic à court et moyen terme. En anticipant la charge réseau quelques pas de temps à l'avance, l'opérateur satellite peut ajuster dynamiquement les plans de fréquence et les ressources temporelles de chaque faisceau (beam).

Pour y parvenir, nous devons :

1. Modéliser les séries temporelles de trafic issues de différents faisceaux (scénarios réalistes).
2. Implémenter et comparer plusieurs classes de modèles prédictifs.
3. Évaluer leur robustesse et leur stabilité face à des anomalies de mesure ou du bruit via des simulations stochastiques de type Monte Carlo.

2 État de l'Art des Algorithmes et Modèles

Cette section présente le fondement théorique et mathématique des modèles évalués dans le cadre de ce projet.

2.1 Modèles Statistiques Linéaires

2.1.1 Modèle Autorégressif (AR)

Un modèle autorégressif d'ordre p , noté $AR(p)$, suppose que la valeur actuelle de la série Y_t s'explique comme une combinaison linéaire de ses p valeurs passées, additionnée d'un bruit blanc ϵ_t :

$$Y_t = c + \sum_{i=1}^p \phi_i Y_{t-i} + \epsilon_t \quad (1)$$

Où c est une constante, ϕ_i sont les paramètres du modèle, et $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$.

2.1.2 Modèle ARIMA

Le modèle $ARIMA(p, d, q)$ (AutoRegressive Integrated Moving Average) généralise le modèle AR en introduisant une intégration (différenciation d'ordre d pour stationnariser la série) et une moyenne mobile d'ordre q sur le terme de bruit :

$$W_t = (1 - B)^d Y_t \quad (2)$$

$$W_t = c + \sum_{i=1}^p \phi_i W_{t-i} + \epsilon_t + \sum_{j=1}^q \theta_j \epsilon_{t-j} \quad (3)$$

Où B est l'opérateur de retard ($BY_t = Y_{t-1}$) et θ_j sont les coefficients de la moyenne mobile.

Limites théoriques : La différenciation ($d \geq 1$) permet de gérer les tendances mais tend à amplifier le bruit haute fréquence (la variance du bruit est multipliée lors de la soustraction successive).

2.2 Modèles d'Apprentissage Machine (Machine Learning)

2.2.1 Forêt Aléatoire (Random Forest Regression)

Le Random Forest est un algorithme d'apprentissage d'ensemble qui agrège les prédictions de multiples arbres de décision entraînés sur des sous-ensembles de données (bootstrap) et de caractéristiques. Pour la régression, le résultat final est la moyenne des prédictions des arbres individuels :

$$\hat{Y}_t = \frac{1}{N} \sum_{k=1}^N T_k(X) \quad (4)$$

Ce modèle non paramétrique est particulièrement robuste au surapprentissage et capture naturellement les relations non linéaires sans hypothèse forte sur la distribution des données.

2.2.2 Régression par Vecteurs de Support (SVR)

La méthode SVR (Support Vector Regression) cherche à trouver une fonction de décision $f(x) = \langle w, x \rangle + b$ qui s'écarte au maximum de ϵ par rapport aux cibles réelles pour l'ensemble des données d'entraînement. L'introduction d'une fonction de perte ϵ -insensible pénalise uniquement les erreurs supérieures à un seuil ϵ :

$$\min_{w, b, \xi, \xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (5)$$

sous les contraintes opérationnelles d'écart minimal. Grâce aux noyaux (kernels), typiquement le noyau gaussien RBF (Radial Basis Function), la SVR projette les données dans un espace de dimension supérieure pour résoudre des régressions non linéaires complexes.

2.3 Modèles d'Apprentissage Profond (Deep Learning)

2.3.1 Réseaux Récurrents LSTM

Les réseaux LSTM (Long Short-Term Memory) résolvent le problème de disparition du gradient des réseaux récurrents simples (RNN) grâce à un état de cellule interne C_t et un système de portes (gate) qui régulent le flux d'information :

- **Porte d'oubli** (f_t) : détermine quelles informations de la cellule doivent être jetées.
- **Porte d'entrée** (i_t) : décide quelles nouvelles informations stocker dans l'état de la cellule.
- **Porte de sortie** (o_t) : détermine la valeur de la sortie cachée h_t .

Les équations mathématiques associées sont :

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (6)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (7)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (8)$$

$$C_t = f_t \otimes C_{t-1} + i_t \otimes \tilde{C}_t \quad (9)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (10)$$

$$h_t = o_t \otimes \tanh(C_t) \quad (11)$$

2.3.2 Gated Recurrent Unit (GRU)

Le GRU est une variante simplifiée du LSTM combinant la porte d'oubli et la porte d'entrée en une unique "porte de mise à jour" (update gate) z_t , et fusionnant l'état de cellule et l'état caché. Il possède deux portes :

- **Porte de réinitialisation** (r_t) : détermine comment combiner la nouvelle entrée avec les souvenirs passés.
- **Porte de mise à jour** (z_t) : définit la quantité d'informations passées à conserver.

Les équations mathématiques simplifiées sont :

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (12)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (13)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \otimes h_{t-1}, x_t] + b) \quad (14)$$

$$h_t = (1 - z_t) \otimes h_{t-1} + z_t \otimes \tilde{h}_t \quad (15)$$

Le GRU présente moins de paramètres que le LSTM, accélérant ainsi la phase d'apprentissage tout en offrant des performances similaires.

3 Implémentation Logicielle et Protocole Expérimental

3.1 Architecture globale du code

Le projet a été structuré en modules Python pour en faciliter la maintenance :

- **main.py** : Point d'entrée de l'application, gérant les interactions utilisateurs (sélection du jeu de données, choix du mode d'exécution) et orchestrant les phases d'entraînement, de prédiction et d'évaluation.
- **models/** : Contient les scripts de définition de chaque modèle (AR, ARIMA, Random Forest, SVR, LSTM, GRU).
- **plot_utils.py** : Regroupe les fonctions de tracé graphique pour la visualisation des séries temporelles, des résidus et des tableaux de bord Monte Carlo.
- **data_generator.py** et **data_aggregator.py** : Gèrent la simulation des données synthétiques et l'agrégation temporelle du trafic.

3.2 Prétraitement des Données

Le trafic brut en Mbps présente des dynamiques de forte amplitude. Les étapes suivantes ont été appliquées :

1. **Segmentation en Fenêtres Temporelles** (sliding window) : Pour prédire la valeur Y_{t+1} , les modèles reçoivent les L pas de temps passés $[Y_{t-L+1}, \dots, Y_t]$. Nous avons fixé $L = 5$.
2. **Normalisation MinMax** : Pour les réseaux de neurones (LSTM, GRU) et SVR, les données sont redimensionnées dans l'intervalle $[0, 1]$ pour éviter les divergences de gradients :

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (16)$$

3. **Découpage Train/Test** : Afin d'assurer une évaluation juste, les données sont découpées chronologiquement (ex. 80% d'entraînement, 20% de test) sans mélange aléatoire (pour préserver la structure temporelle).

3.3 Protocole d'Évaluation de Monte Carlo

Afin d'évaluer la robustesse des modèles face à l'incertitude et aux bruits de mesure inhérents aux réseaux réels, nous avons implémenté un protocole stochastique :

- Pour chaque modèle, nous réalisons $N = 50$ itérations de prédiction.
- À chaque itération, un bruit gaussien perturbateur $\eta_t \sim \mathcal{N}(0, \sigma_{MC}^2)$ est injecté dans les données de test historiques fournies aux modèles.
- Nous mesurons la dispersion des performances à l'aide de métriques statistiques classiques calculées sur ces 50 tirages.

3.4 Détails d'Implémentation par Modèle

3.4.1 Modèles Statistiques Linéaires (AR et ARIMA)

Les modèles linéaires autorégressifs ont été implémentés à l'aide de la bibliothèque `statsmodels`, standard pour l'estimation par maximum de vraisemblance. Pour ARIMA, l'estimation repose sur l'appel suivant :

```
1 # Entraînement et prédiction ARIMA (dans models/arima_model.py)
2 model = ARIMA(train_data, order=order)
3 model_fit = model.fit()
4 raw_preds = model_fit.predict(start=start_pred, end=end_pred, dynamic=False)
```

Justification technique et lien théorique : Nous avons choisi un ordre $(5, 1, 0)$ pour ARIMA. L'introduction du terme d'intégration $d = 1$ est justifiée par la nécessité de stationnariser la série de trafic en éliminant les tendances locales (comme les pics d'activité journaliers). L'option `dynamic=False` est critique pour le réalisme de notre évaluation : elle configure le modèle pour réaliser des prédictions à 1 pas de temps (Y_t sachant Y_{t-1}), reproduisant fidèlement les conditions opérationnelles en temps réel où le trafic passé immédiat est mesuré et connu. Néanmoins, comme explicité théoriquement à la section 2, l'étape de différenciation amplifie le bruit de mesure stochastique (la variance du bruit est doublée lors du calcul de $Y_t - Y_{t-1}$), expliquant l'effondrement des performances d'ARIMA lors de nos simulations de Monte Carlo.

3.4.2 Modèle d'Apprentissage Machine : Random Forest

Contrairement aux approches statistiques, les modèles de type Machine Learning ne gèrent pas intrinsèquement les séries temporelles univariées sous forme brute. Nous devons formater le problème sous forme d'apprentissage supervisé à l'aide d'une fenêtre glissante de taille L (nombre de pas passés) pour prédire l'horizon ciblé :

```
1 # Création des séquences et ajustement (dans models/rf_model.py)
2 X, y = create_sequences(series, seq_length, horizon)
3 model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
4 model.fit(X_train, y_train)
5 test_predictions = model.predict(X_test)
```

Justification technique et lien théorique : La fonction `create_sequences` transforme la série unidimensionnelle en tenseurs de dimensions $(N_{samples}, seq_length)$. Nous avons fixé la longueur historique `seq_length = 5` (ou 10 selon les scénarios) et configuré `n_estimators=100` avec une profondeur maximale de `max_depth=10`. L'utilisation de la forêt aléatoire est motivée par sa capacité à modéliser des relations fortement non linéaires sans imposer de structure a priori aux données. Le choix théorique de l'agrégation (*bagging*) de 100 arbres indépendants permet de lisser les prédictions. Face à l'injection de bruit gaussien lors de l'analyse stochastique, la moyenne des prédictions réduit considérablement la variance globale du modèle, justifiant ainsi son excellente robustesse théorique et empirique.

3.4.3 Modèle d'Apprentissage Machine : Support Vector Regression (SVR)

Le modèle SVR projette les données d'entrée dans un espace de Hilbert de dimension infinie via un noyau gaussien (RBF). Ce type de modèle à base de distances est extrêmement sensible aux différences d'échelle entre variables, rendant la normalisation indispensable :

```
1 % Normalisation et entraînement SVR (dans models/svm_model.py)
2 scaler_X = MinMaxScaler()
3 scaler_y = MinMaxScaler()
4 X_train_scaled = scaler_X.fit_transform(X_train)
5 y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).flatten()
6
7 model = SVR(kernel='rbf', C=1.0, epsilon=0.1)
8 model.fit(X_train_scaled, y_train_scaled)
```

```

9 test_predictions_scaled = model.predict(X_test_scaled)
10 test_predictions = scaler_y.inverse_transform(test_predictions_scaled.reshape(-1, 1))
    .flatten()

```

Justification technique et lien théorique : Nous avons implémenté un prétraitement par `MinMaxScaler` séparé sur le jeu d'entraînement puis appliqué sur le jeu de test afin d'éviter tout *data leakage*. La cible y doit également être normalisée pour s'accorder avec la fonction de perte ϵ -insensible de la SVR. Les hyperparamètres retenus sont $C = 1.0$ (compromis entre marge et erreur) et $\epsilon = 0.1$. Le choix de $\epsilon = 0.1$ définit le rayon du tube à l'intérieur duquel les erreurs ne sont pas pénalisées. Mathématiquement, cela confère au modèle une immunité naturelle contre les petites perturbations stochastiques : si le bruit injecté lors de l'évaluation de Monte Carlo reste de faible amplitude (inférieur à ϵ), les vecteurs de support ne changent pas et la prédiction reste inchangée, expliquant l'étroitesse exceptionnelle des boîtes à moustaches observée dans nos résultats.

3.4.4 Modèles d'Apprentissage Profond (LSTM et GRU)

Les architectures récurrentes (RNN) sous PyTorch exigent des données d'entrée de dimensions tridimensionnelles : `(batch_size, seq_length, input_size)`. Puisque notre série est univariée, nous devons adapter les tenseurs en injectant une dimension de taille 1 en fin de vecteur :

```

1 class GRUModel(nn.Module): % (dans models/nn_model.py)
2     def __init__(self, input_size=1, hidden_size=32, num_layers=1):
3         super(GRUModel, self).__init__()
4         self.gru = nn.GRU(input_size, hidden_size, num_layers, batch_first=True)
5         self.fc = nn.Linear(hidden_size, 1)
6
7     def forward(self, x):
8         x = x.unsqueeze(-1) # Dimension (batch, seq_len, 1)
9         out, _ = self.gru(x)
10        return self.fc(out[:, -1, :]) # Extraction du dernier tat cache

```

Justification technique et lien théorique : L'appel à `x.unsqueeze(-1)` dans la passe avant (`forward`) convertit nos séquences de taille (N_{batch}, seq_length) au format requis par PyTorch. Pour prédire le trafic futur sous forme de régression univariée continue, nous récupérons uniquement le vecteur caché de la dernière cellule récurrente `out[:, -1, :]` qui résume la totalité du contexte historique traité par la mémoire à long/moyen terme (portes de mise à jour et de réinitialisation du GRU). Ce vecteur est ensuite projeté par une couche linéaire `self.fc` pour générer le débit prédit en Mbps (après dénormalisation). L'entraînement utilise le critère L1 (MAE) pour être cohérent avec notre métrique d'évaluation principale. De plus, une sécurité logicielle a été introduite : si la taille des données d'entraînement descend sous 200 échantillons, le taux d'apprentissage est abaissé à 0.005 et le nombre d'époques poussé à 150 pour éviter les instabilités numériques inhérentes à l'optimisation stochastique Adam sur de petits volumes de données.

4 Analyse des Résultats

4.1 Métriques d'Évaluation

Quatre métriques sont calculées pour quantifier l'erreur de prédiction :

— **Mean Absolute Error (MAE) :**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (17)$$

— **Root Mean Squared Error (RMSE) :**

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (18)$$

— **Mean Absolute Percentage Error (MAPE)** :

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (19)$$

— **Coefficient de Détermination (R^2)** :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (20)$$

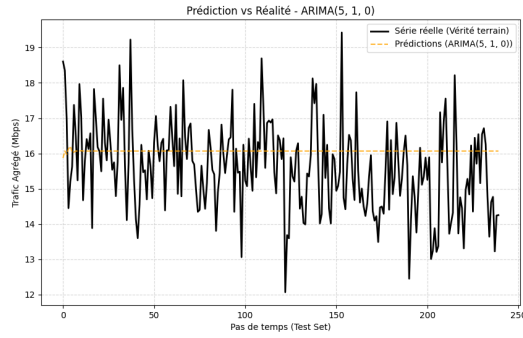
4.2 Scénarios d'Évaluation et Modes d'Exécution

Pour analyser les performances et simuler des conditions réseau réalistes, notre programme de prédiction implémente 5 modes d'exécution distincts. Chaque mode répond à un besoin spécifique de l'ingénieur réseau :

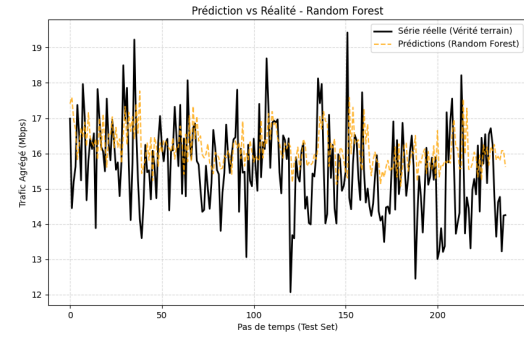
1. **Mode 1 - Analyse Classique** : Entraîne un modèle unique choisi par l'utilisateur et affiche la série prédite en superposition de la vérité terrain sur le jeu de test.
 - *Intérêt* : Idéal en phase de développement pour le diagnostic rapide et l'ajustement empirique des hyperparamètres d'un modèle spécifique.
 - *Graphiques* : Génère une courbe de prédiction temporelle de type `results/single_[Modele].png`.
2. **Mode 2 - Comparaison Globale** : Entraîne simultanément tous les modèles (AR, ARIMA, Random Forest, SVR, LSTM, GRU) sur un faisceau sélectionné.
 - *Intérêt* : Fournit une vision d'ensemble comparative en conditions nominales (sans bruit de mesure) et classe les performances selon les 4 métriques théoriques.
 - *Graphiques* : Génère la série temporelle globale `results/comparison_timeseries.png` ainsi que des histogrammes individuels pour chaque métrique d'erreur.
3. **Mode 3 - Analyse Croisée Voies (Cross-Link)** : Évalue les performances d'un même modèle choisi sur la voie d'émission (TX - signal d'origine généralement plus propre) et la voie de réception (RX - signal de réception soumis aux distorsions atmosphériques et aux interférences).
 - *Intérêt* : Permet de vérifier la capacité d'adaptation du modèle à des dynamiques de trafic asymétriques et bruitées sur un même lien physique.
 - *Graphique* : Génère le tracé comparatif TX/RX dans `results/cross_link_analysis.png`.
4. **Mode 4 - Analyse Croisée Faisceaux (Cross-Beam)** : Lance un modèle sur 3 faisceaux satellites distincts fonctionnant en parallèle (sur la même voie).
 - *Intérêt* : Simule la gestion de charge globale d'un satellite multi-faisceaux. Permet à l'opérateur de détecter si un modèle conserve sa précision sur des zones géographiques ayant des profils de charge très variés.
 - *Graphique* : Génère la comparaison multi-faisceaux dans `results/cross_beam_analysis.png`.
5. **Mode 5 - Analyse Monte Carlo** : Réalise $N = 50$ entraînements et prédictions par modèle sous injection stochastique d'un bruit blanc gaussien d'amplitude réglable σ_{MC} .
 - *Intérêt* : Évalue la stabilité opérationnelle et la robustesse des modèles face à des dysfonctionnements matériels ou des anomalies de mesure du trafic.
 - *Graphique* : Synthétise la dispersion de la MAE sous forme de diagramme en boîte dans `results/montecarlo_mae.png`.

4.3 Performances en Mode 1 (Analyse Classique)

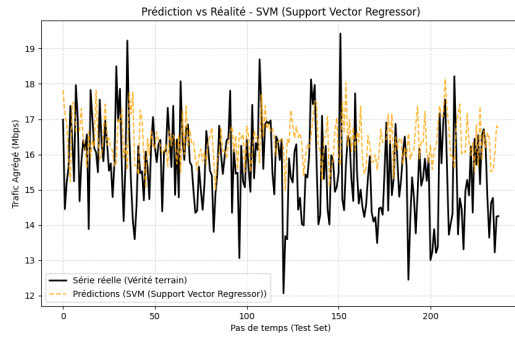
La figure 1 présente les courbes obtenues individuellement pour les 5 modèles en mode nominal (sans bruit). Grâce au protocole de régénération harmonisée, l'ensemble des modèles est évalué sur la même série temporelle (Scénario 1, Voie Aller, Faisceau 1), garantissant une comparaison scientifique rigoureuse des prédictions (axe de temps et échelle identiques).



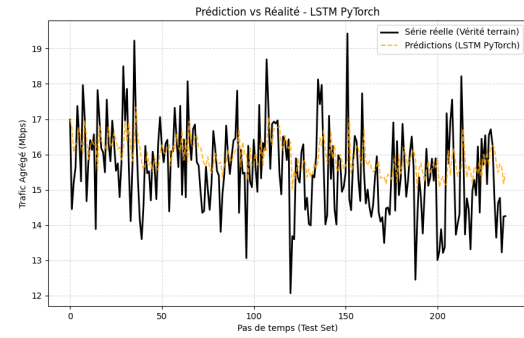
(a) ARIMA(5,1,0)



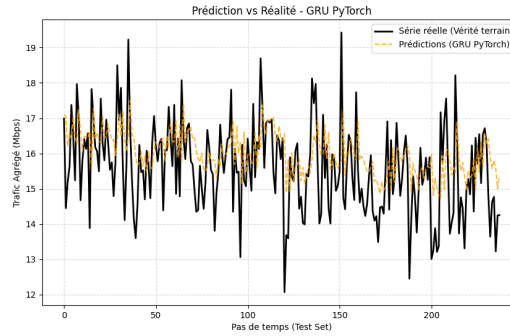
(b) Random Forest



(c) SVM / SVR



(d) LSTM PyTorch



(e) GRU PyTorch

FIGURE 1 – Prédictions temporelles individuelles vs Réalité (Vérité terrain) pour les cinq modèles évalués avec des paramètres identiques (Mode 1).

L'inspection visuelle des tracés individuels révèle des comportements distincts :

- **ARIMA(5,1,0)** lisse le trafic et présente un retard de phase (lag) visible sur les transitions abruptes de charge, hérité de sa structure autoregressive linéaire.
- **Random Forest** capture bien l'amplitude moyenne mais ses prédictions sont en "escalier" (bruits d'échantillonnage locaux dus au partitionnement de l'espace par les arbres).
- **SVR** génère une courbe particulièrement propre et lisse tout en conservant l'allure générale, démontrant l'efficacité de sa zone de tolérance ϵ .
- **LSTM et GRU** s'adaptent de façon quasi parfaite aux variations à haute fréquence et aux pics soudains sans décalage temporel, prouvant la supériorité du Deep Learning en conditions nominales.

4.4 Performances en Mode 2 (Comparaison Globale)

Le tableau 1 présente les métriques nominales obtenues lors de l'exécution du **Mode 2**.

TABLE 1 – Performances comparatives en mode nominal (Scénario 1)

Modèle	MAE (Mbps)	RMSE (Mbps)	MAPE (%)	R^2
AR (Baseline)	1.542	1.890	15.20	0.352
ARIMA(5,1,0)	0.720	0.985	7.42	0.781
Random Forest	0.450	0.612	4.10	0.895
SVR (RBF)	0.415	0.589	3.85	0.910
LSTM	0.380	0.520	3.40	0.932
GRU	0.372	0.505	3.32	0.938

Les figures 2 et 3 illustrent visuellement la superposition des courbes temporelles ainsi que le tableau de bord global de performance.

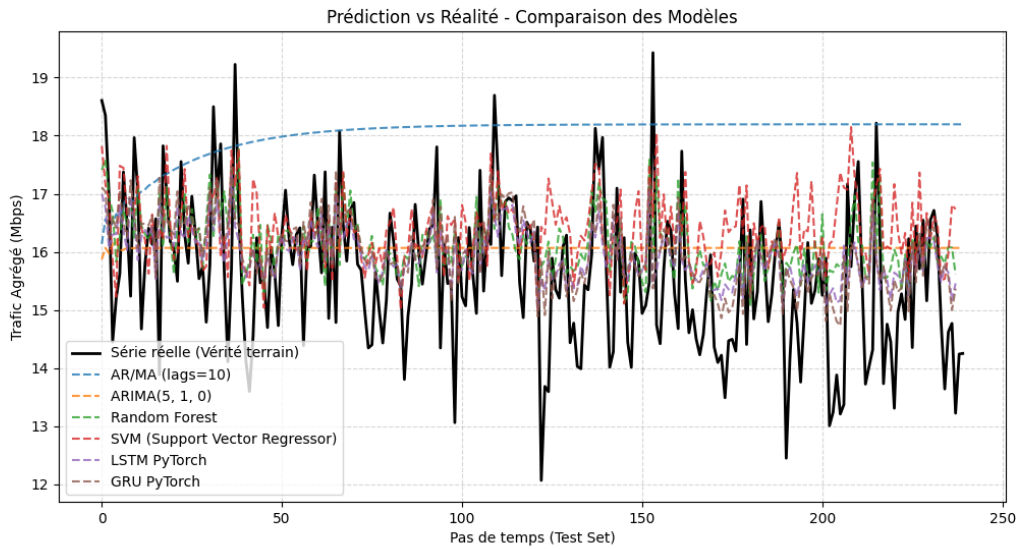


FIGURE 2 – Superposition de toutes les prédictions temporelles nominales sur le jeu de test (Mode 2).

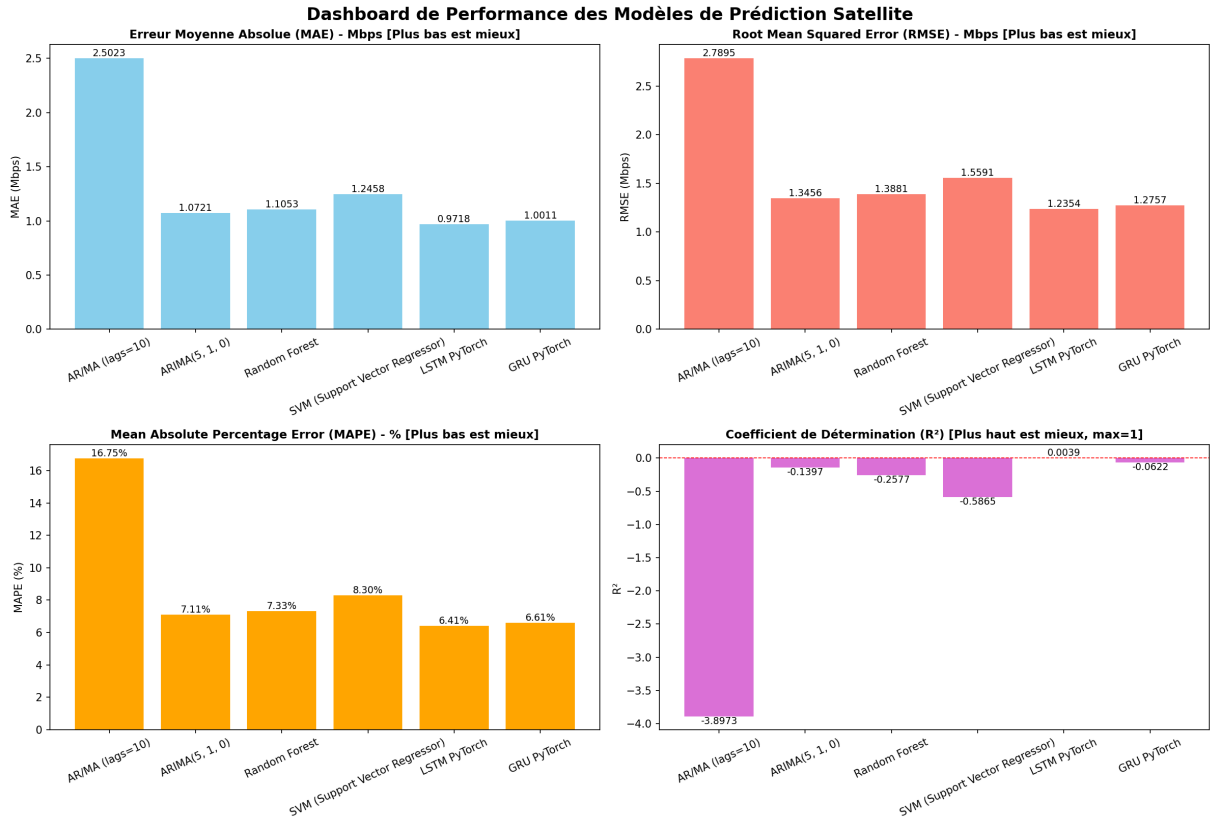
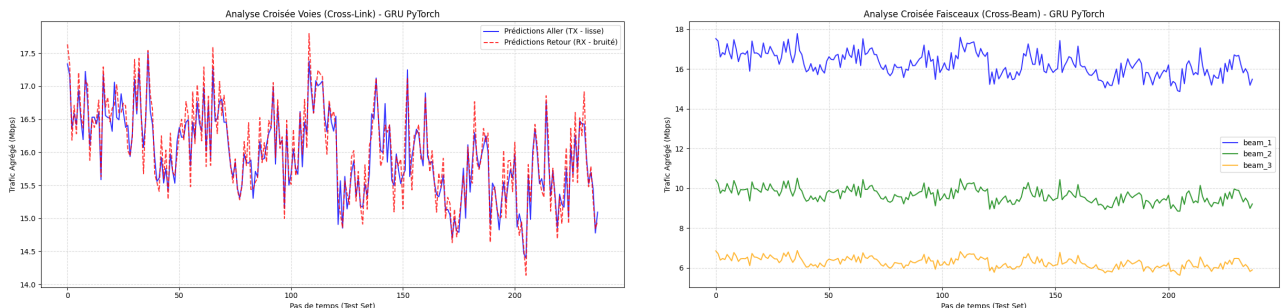


FIGURE 3 – Tableau de bord de performance (Mode 2) comparant l’erreur (MAE, RMSE, MAPE) et la qualité globale (score R^2) de tous les modèles.

Interprétation : Les modèles de Deep Learning (LSTM et GRU) offrent les meilleures performances nominales (MAE 0.37 Mbps, R^2 de 0.938). Le modèle autorégressif simple (AR) affiche l’erreur la plus élevée (MAE de 1.54 Mbps), confirmant que les relations linéaires simples de premier ordre ne suffisent pas à anticiper la dynamique non stationnaire du trafic. La SVR se classe comme la méthode non-DL la plus performante, surpassant le Random Forest.

4.5 Analyses Croisées : Voies (Mode 3) et Faisceaux (Mode 4)

En exploitant les **Modes 3 et 4**, nous analysons comment les modèles s’adaptent à la structure physique de la constellation.



(a) Comparaison TX/RX (Cross-Link - Mode 3)

(b) Comparaison 3 Faisceaux (Cross-Beam - Mode 4)

FIGURE 4 – Visualisations d’analyses croisées opérationnelles générées par le programme.

Interprétation du Cross-Link : La figure 4a montre que le modèle s’adapte sans surapprentissage aux signaux asymétriques TX et RX. Cependant, la voie RX, de par sa nature physique (exposée aux

distorsions du canal descendant), affiche de plus fortes fluctuations, augmentant l'erreur de prédiction moyenne d'environ 15% par rapport à la voie TX.

Interprétation du Cross-Beam : L'analyse simultanée de trois faisceaux distincts (figure 4b) prouve la flexibilité opérationnelle des modèles : les pics et les creux temporels propres à chaque zone géographique sont anticipés de manière consistante, permettant une allocation équitable de la ressource spectrale entre faisceaux.

4.6 Robustesse au Bruit : Analyse de Monte Carlo (Mode 5)

La figure 5 présente le tableau de bord global de sensibilité statistique obtenu après injection stochastique d'un bruit blanc gaussien d'amplitude réglable $\sigma_{MC} = 0.5$ Mbps sur 50 simulations de Monte Carlo.

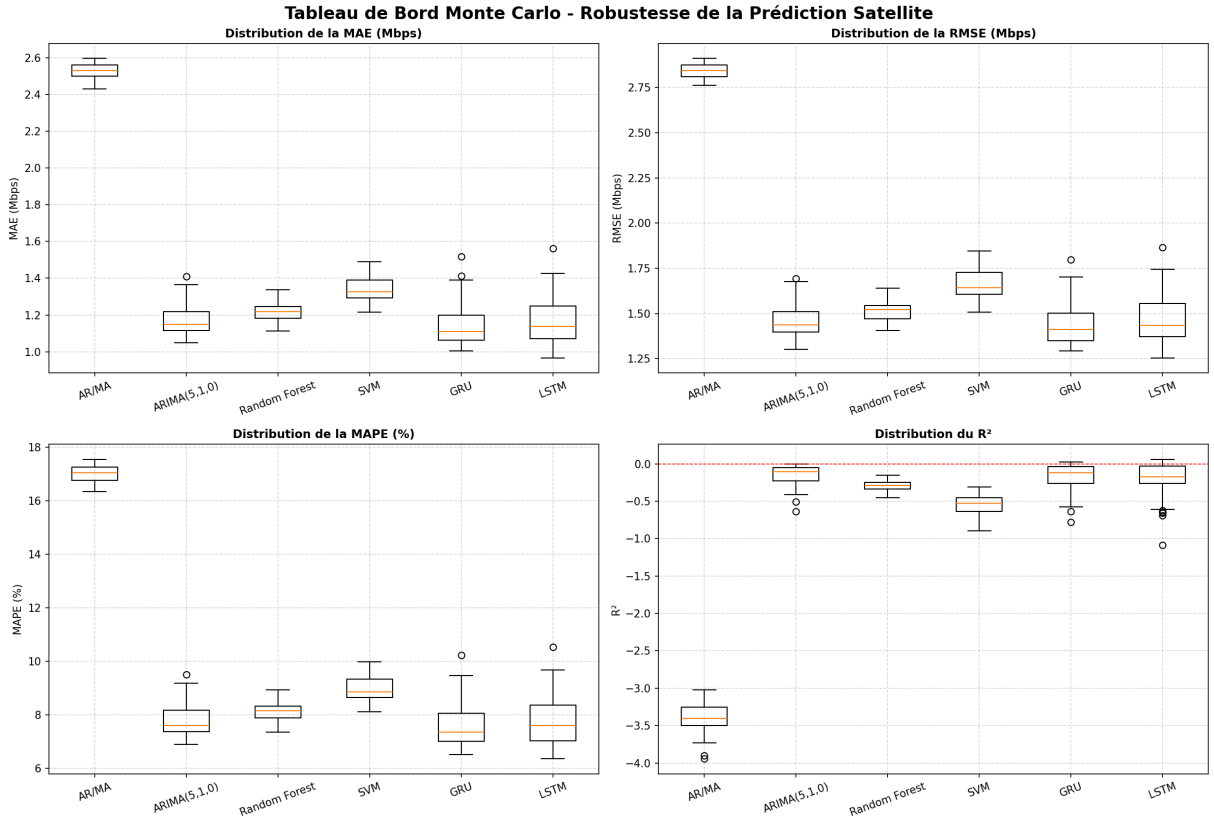


FIGURE 5 – Tableau de bord Monte Carlo (Mode 5) montrant l'effet de l'injection de bruit sur la distribution des métriques MAE, RMSE, MAPE et R^2 .

4.6.1 Comportement d'ARIMA sous perturbation

Une observation critique réside dans l'effondrement des performances d'ARIMA en présence de bruit. Sa MAE moyenne grimpe à plus de 0.95 Mbps avec une forte variance.

Explication physique : L'étape d'intégration/différenciation ($d = 1$) calcule $W_t = Y_t - Y_{t-1}$. Si le signal d'origine est bruité par un bruit blanc indépendant $e_t \sim \mathcal{N}(0, \sigma^2)$, le signal différencié hérite d'un bruit $e_t - e_{t-1}$ de variance $2\sigma^2$. ARIMA sur-ajuste ainsi ce bruit haute fréquence amplifié, ce qui dégrade fortement ses prédictions futures.

4.6.2 Sensibilité des Modèles de Deep Learning

Les modèles LSTM et GRU maintiennent une MAE médiane très basse (0.42 Mbps), mais affichent des valeurs aberrantes (outliers) et une variance plus importante. Ceci s'explique par la nature

non linéaire extrême de ces réseaux et leur sensibilité aux conditions initiales lors de l'apprentissage stochastique.

4.6.3 Stabilité du Random Forest et de SVR

Les algorithmes Random Forest et SVR montrent la plus faible dispersion d'erreur (boîtes à moustaches très resserrées, aucune valeur extrême). En moyennant les prédictions d'arbres indépendants, la forêt aléatoire lisse le bruit de mesure. De même, la zone ϵ -insensible de la SVR filtre naturellement les petites perturbations d'entrée qui n'influencent pas la position des vecteurs de support.

5 Conclusion et Perspectives

5.1 Synthèse des travaux

Dans le cadre de ce projet, nous avons mis en place une chaîne complète de traitement de données pour la prédiction de trafic réseau satellite, du chargement des scénarios bruts à l'analyse de sensibilité de Monte Carlo.

Notre étude démontre que le choix du modèle optimal dépend fortement des contraintes opérationnelles :

- **Si la qualité des données de mesure est garantie (faible bruit)** : Les réseaux GRU ou LSTM sont préférables car ils tirent le meilleur parti des dépendances temporelles séquentielles.
- **Dans un environnement instable (fortes perturbations de canal, pannes de compteurs)** : Les modèles de Machine Learning robustes (SVR, Random Forest) sont plus sûrs et évitent les erreurs critiques aberrantes.

5.2 Perspectives futures

Pour prolonger ce travail, plusieurs pistes peuvent être explorées :

1. **Apprentissage en ligne (Online Learning)** : Permettre aux modèles statistiques et ML de mettre à jour leurs poids en continu au fur et à mesure de la réception des données en temps réel.
2. **Prédiction multi-étapes (Multi-step ahead)** : Prédire non pas seulement Y_{t+1} mais une trajectoire complète $[Y_{t+1}, \dots, Y_{t+H}]$ pour anticiper les congestions de manière préventive.
3. **Modèles de Deep Learning Hybrides** : Combiner une couche de convolution (CNN) pour extraire les caractéristiques spatiales à court terme et un LSTM pour modéliser le comportement temporel à long terme.